

SECURE HASH ALGORITHM

A SEARIES OF SHA...

Secure Hash Algorithm

- SHA originally designed by NIST & NSA in 1993
- was revised in 1995 as SHA-1
- US standard for use with DSA signature scheme
 - ▣ standard is FIPS 180-1 1995, also Internet RFC3174
 - ▣ nb. the algorithm is SHA, the standard is SHS
- based on design of MD5 with key differences
- produces 160-bit hash values
- recent 2005 results on security of SHA-1 have raised concerns on its use in future applications

How SHA Works?

- Digest Length=**160 bit**
- I/P Text=**512 bit**
- Sub Block size=**32bit**
- $512/32=16$ total Sub blocks
- No. Of Rounds=**4**
- Iteration per round=**20**
- Chaining Variable = $5*32=160$
- $K[t]$ constant= **Where $t=0$ to 79**
- O/P-> **four 32 bit blocks**

SHA Overview

1. **Padding**: Length of the message is 64 bits short of multiple of 512 after padding.
2. **Append** a 64-bit **length** value of original message is taken.
3. **Divide the input into 512-bit blocks**
4. **Initialise CV** 5-word (160-bit) buffer (A,B,C,D,E) to
(A=01 23 45 67,
B=89 AB CD EF,
C=FE DC BA 98,
D=76 54 32 10,
E=C3 D2 E1 F0)

Continue...

5. **Process Blocks** now the actual algorithm begins.
message in 16-word (512-bit) chunks:

- Copy CV into single register for storing temporary intermediate as well as the final results.
- Divide the current 512-bit blocks into 16 sub-blocks, each consisting of 32 bits.
- Has No. Of Rounds=4, each round consisting of 20 bit /step iteration operations on message block & buffer
- expand 16 words into 80 words(20×4) by mixing & shifting. $K[t]$ constant= Where $t=0$ to 79
- Form new buffer value by adding output to input.

6. output hash value is the final buffer value

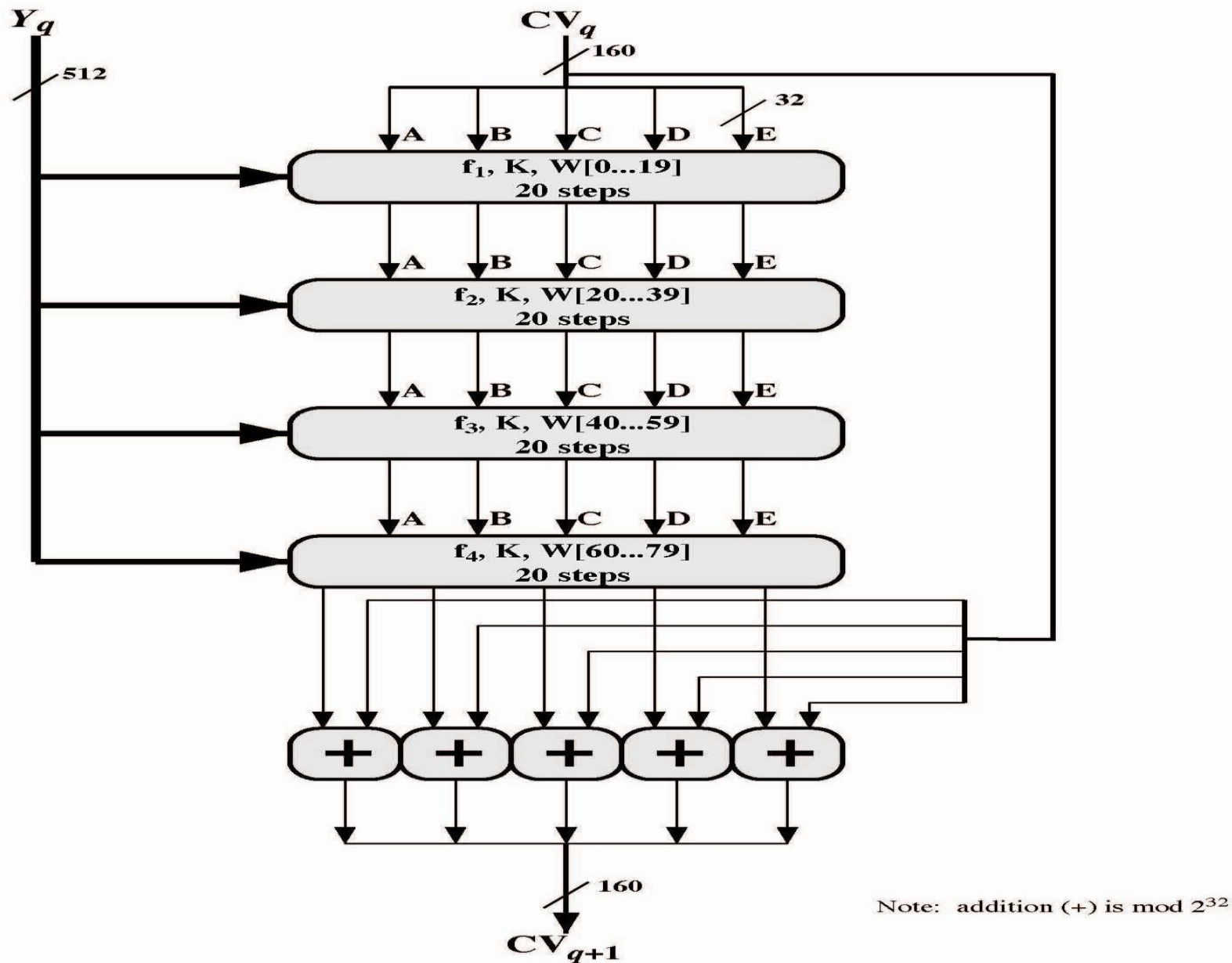
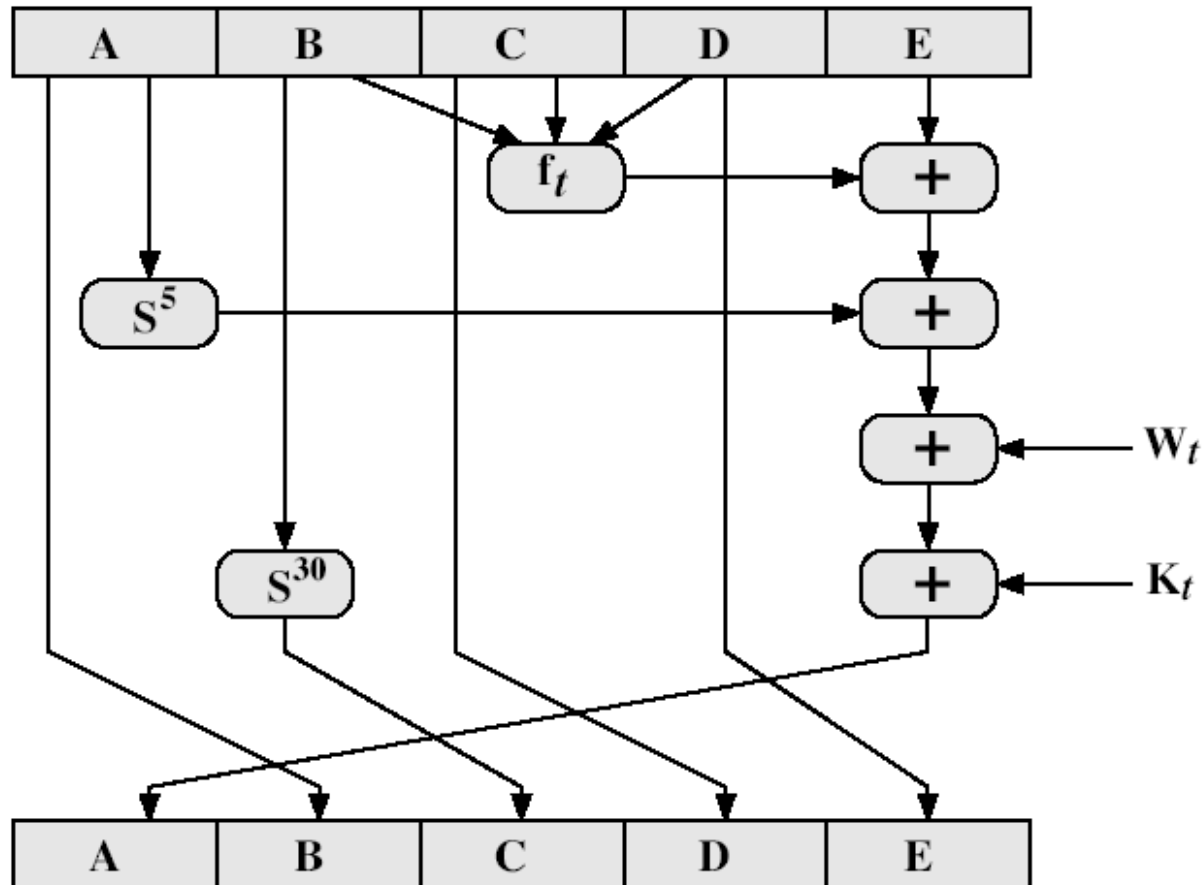


Figure 12.5 SHA-1 Processing of a Single 512-bit Block (SHA-1 Compression Function)

SHA-1 Compression Function



$ABCDE = (F[t] + E + S^5(A) + W[t] + K[t]), \gg \gg$ Shift right by 1 bit for next iteration

SHA-1 Compression Function terms

- each round has 20 steps which replaces the 5 buffer words thus:

$$(A, B, C, D, E) \leftarrow (E + f(t, B, C, D) + (A \ll 5) + W_t + K_t), A, (B \ll 30), C, D)$$

- ABCDE refer to the 5 words of the buffer
- t is the step number
- $f(t, B, C, D)$ is nonlinear function for round
- W_t is derived from the message block
- K_t is a constant value
- S^t circular left shift of 32 bit sub-block by t bits

Process $F(t)$ in each SHA-1 round

□ where g can be expressed as:

ROUND 1: $(b \text{ AND } c) \text{ OR } ((\text{NOT } b) \text{ AND } (d))$ same as MD5

ROUND 2: $b \text{ XOR } c \text{ XOR } d$

ROUND 3: $(b \text{ AND } c) \text{ OR } (b \text{ AND } d) \text{ OR } (c \text{ AND } d)$

ROUND 4: $b \text{ XOR } c \text{ XOR } d$

Creation of 80-word input W_t

- Adds redundancy and interdependence among message blocks

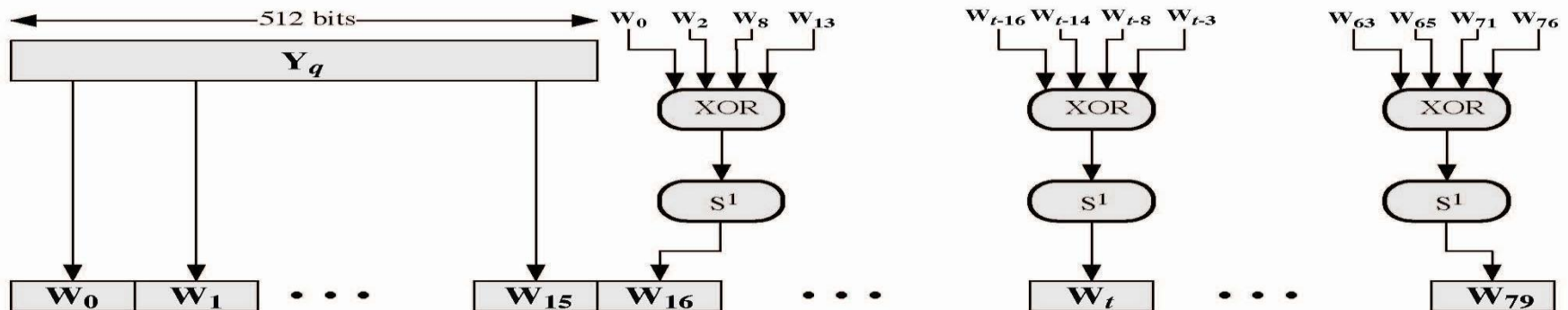


Figure 12.7 Creation of 80-word Input Sequence for SHA-1 Processing of Single Block

SHA-1 verses MD5

- brute force attack is harder (160 vs 128 bits for MD5)
- not vulnerable to any known attacks (compared to MD4/5)
- a little slower than MD5 (80 vs 64 steps)
- both designed as simple and compact
- optimised for big endian CPU's (SUN) vs MD5 for little endian CPU's (PC)

Revised Secure Hash Standard

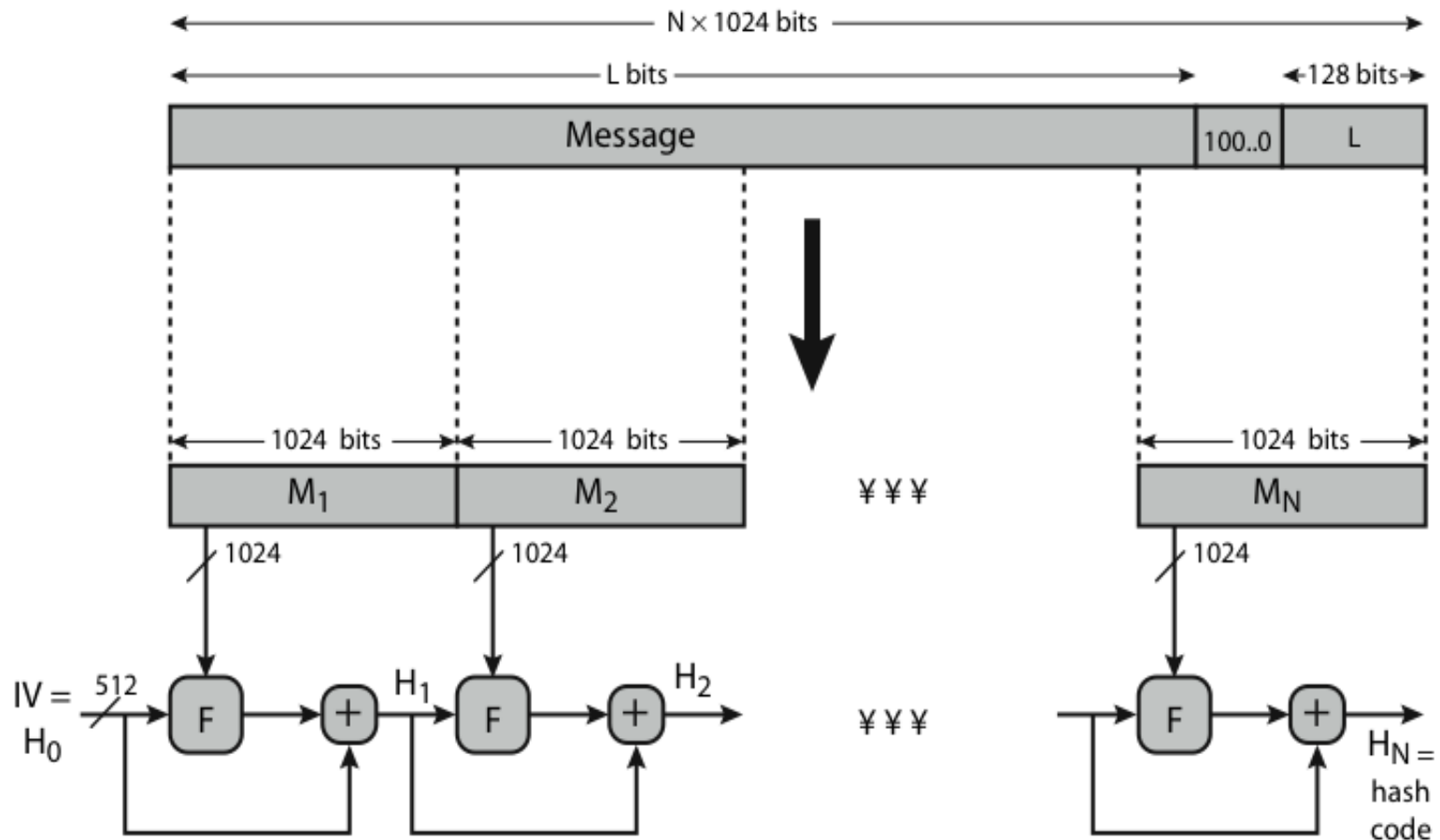
- NIST issued revision FIPS 180-2 in 2002
- adds 3 additional versions of SHA
 - ▣ SHA-256, SHA-384, SHA-512
 - ▣ Different lengths of Message Digest in bits
- designed for compatibility with increased security provided by the AES cipher
- structure & detail is similar to SHA-1
- hence analysis should be similar
- but security levels are rather higher

Table 12.3 Comparison of SHA Properties

	SHA-1	SHA-256	SHA-384	SHA-512
Message digest size	160	256	384	512
Message size	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$
Block size	512	512	1024	1024
Word size	32	32	64	64
Number of steps	80	80	80	80
Security	80	128	192	256

Notes: 1. All sizes are measured in bits.
2. Security refers to the fact that a birthday attack on a message digest of size n produces a collision with a workfactor of approximately $2^{n/2}$.

1. Padding, 2. Append Length, 3. Divide the input into 1024-bit blocks



$+$ = word-by-word addition mod 2^{64}

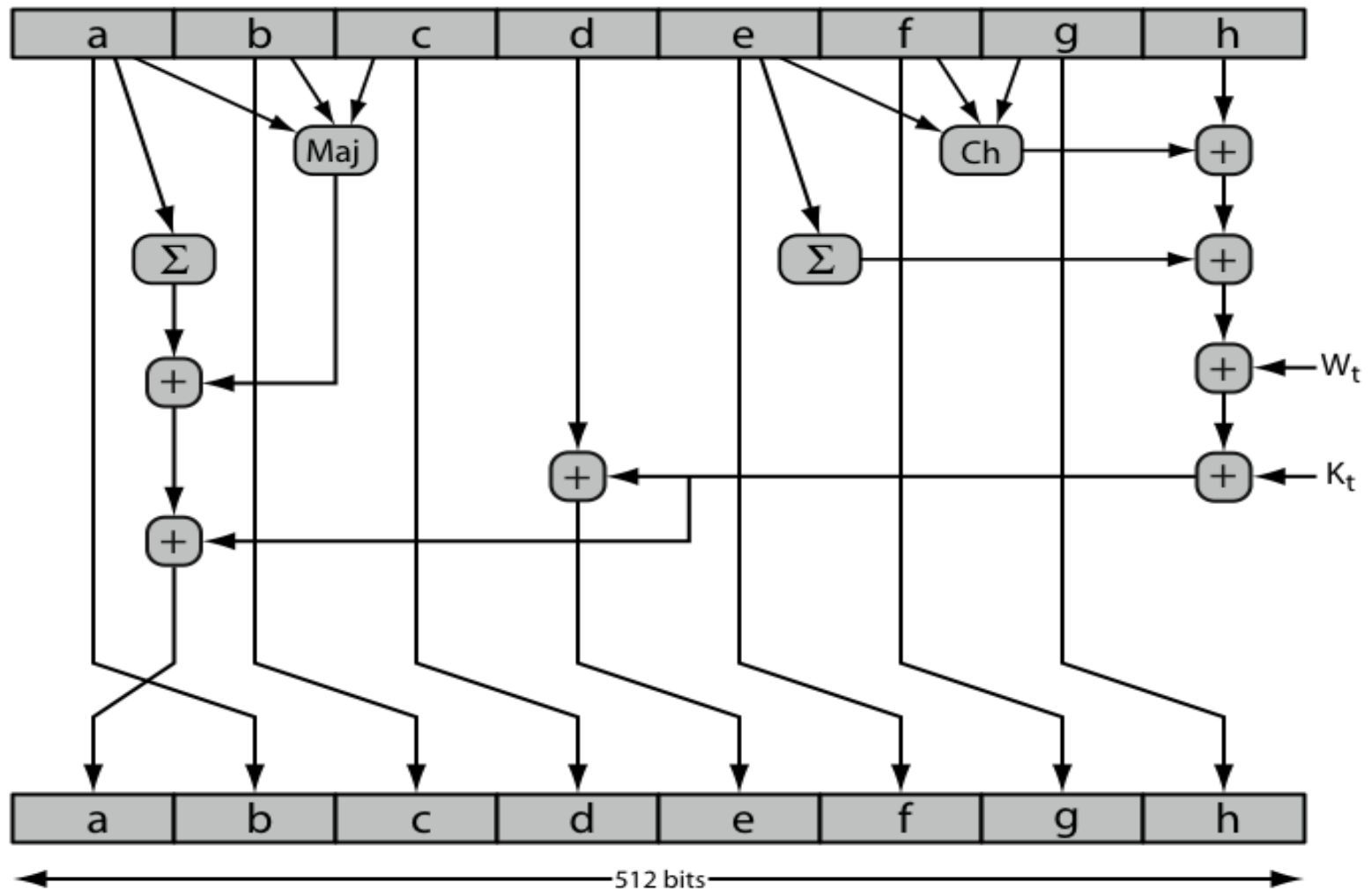
4. Initialization Of Chaining Variable

- $8 * 64 = 512$ bits
- A,B,C,D,E,F,G,H

5.Process Blocks

- heart of the algorithm
- processing message in 1024-bit blocks
- consists of 80 rounds
 - updating a 512-bit buffer TEMP CHAINING VARIABLES.
 - using a 64-bit value W_t derived from the current message block
 - and a round constant based on cube root of first 80 prime numbers

SHA-512 Round Function



SHA-512 Round Function

- Let us look in more detail at the logic in each of the 80 steps of the processing of one 512-bit block (Figure). Each round is defined by the following set of equations:

$$T_1 = h + \text{Ch}(e, f, g) + \left(\sum_1^{512} e \right) + W_t + K_t$$

$$T_2 = \left(\sum_0^{512} a \right) + \text{Maj}(a, b, c)$$

$$a = T_1 + T_2$$

$$b = a$$

$$c = b$$

$$d = c$$

$$e = d + T_1$$

$$f = e$$

$$g = f$$

$$h = g$$

SHA-512 Round Function

- where
- $t = \text{step/round number; } 0 \leq t \leq 79$
- $\text{Ch}(e, f, g) = (e \text{ AND } f) \text{ XOR } (\text{NOT } e \text{ AND } g)$
the conditional function: If e then f else g
- $\text{Maj}(a, b, c) = (a \text{ AND } b) \text{ XOR } (a \text{ AND } c) \text{ XOR } (b \text{ AND } c)$
the function is true only if the majority (two or three) of the arguments are true.
- $\text{Sum}(a_i) = \text{RORT}(a_i \text{ By } 28 \text{ Bit}) \text{ XOR } \text{RORT}(a_i \text{ By } 34 \text{ Bit}) \text{ XOR } \text{RORT}(a_i \text{ By } 39 \text{ Bit})$
- $\text{Sum}(e_i) = \text{RORT}(e_i \text{ By } 14 \text{ Bit}) \text{ XOR } \text{RORT}(e_i \text{ By } 18 \text{ Bit}) \text{ XOR } \text{RORT}(e_i \text{ By } 41 \text{ Bit})$
- $\text{ROTR}_n(x) = \text{circular right shift (rotation) of the 64-bit argument } x \text{ by } n \text{ bits}$
- $W_t = \text{a 64-bit word derived from the current 512-bit input block (i.e.: Message Digest)}$
- $K_t = \text{a 64-bit additive constant}$
- $+$ = addition modulo 2^{64}

SHA-512 Round Function

